

Causal Uplift Agent

for Customer Retention

Code Guide

What each part does, why it is there, and how it works

How to use this guide

This guide explains the code. The *Operation Manual* explains how to install and run it. Read that one first if you have not run the project yet.

Every Python module starts with a line such as `Code guide: section "Metrics" (sec:metrics)`. That is the section of this guide that explains the module. In the other direction, every code listing here is printed from the real source file when the guide is built, with the file name and line numbers in the listing title. The build script (`docs/tools/build_manually.py`) stops with an error if a function shown here no longer exists, so the guide and the code cannot drift apart.

If you want	Read
The idea in 20 minutes	Part I
How the uplift numbers are produced	Part II
How the agent decides who gets an offer	Part III
How it is tested, deployed, and rolled back	Part IV
One customer from data to offer, and the limits	Part V

Contents

I	The big picture	5
1	The business problem	5
2	Uplift in five ideas	5
3	System map	6
4	Repository map	6
5	Results at a glance	6
II	The causal core	7
6	Configuration	8
7	Simulation	8
7.1	Customers	8
7.2	Churn without the offer	9
7.3	The effect of the offer	9
7.4	Assignment, outcome, value	10
8	Data store	11
9	Experiment analysis	11

9.1	Sample ratio mismatch	11
9.2	Covariate balance	11
9.3	Average effect	12
9.4	When assignment is not random	13
10	Uplift learners	14
10.1	Baselines	14
10.2	T-learner	14
10.3	X-learner	14
10.4	Causal forest	15
10.5	IPW learner	16
10.6	Regularization	16
11	Metrics	17
11.1	PEHE	17
11.2	Qini	17
11.3	True Qini	17
11.4	Ties	18
11.5	Doubly robust score	18
12	Bootstrap	18
13	Benchmark	19
14	Estimator selector	20
15	Targeting policy	22
16	Real datasets	22
17	Scorer bundle	25
III	The agent system	25
18	Agent state	25
19	Agent graph	25
20	Memory agent	26
21	SQL agent	27
22	ML agent	27

23 RAG agent	28
24 Policy agent	28
25 Reviewer agent	29
26 Report	30
27 Providers	30
28 SQL service	31
29 Knowledge retrieval	31
30 Memory	32
IV Engineering	32
31 Pipeline	32
32 Quality gates	33
33 MLflow registry	33
34 API	34
35 UI	35
36 Docker	35
37 Airflow	36
38 CI	36
39 Agent evaluation	37
V Putting it together	38
40 One customer's journey	38
41 Limits and next steps	38
42 Self-check questions	39
A Glossary	39

B	References	39
C	Commands	40

Part I

The big picture

1 The business problem

A subscription company loses about 12% of its customers every 30 days. The retention team can send an offer (a check-in message plus a 20% discount on one monthly fee) to some customers. Offers cost money, and the budget covers only part of the base.

The usual approach is to rank customers by churn risk and contact the riskiest ones. That answers the wrong question. Risk says who will leave. The team needs to know who will *stay because of the offer*. Those are different people:

Type	Behaviour	Should we contact?
Persuadable	Leaves without the offer, stays with it	Yes. This is the only group that pays back.
Sure thing	Stays either way	No. The discount is wasted.
Lost cause	Leaves either way	No. Nothing changes.
Sleeping dog	Stays without contact, leaves when reminded	Never. Contact causes churn.

Risk ranking puts lost causes at the top of the list and cannot see sleeping dogs. Uplift modeling estimates, for each customer, how much the offer changes their chance of churning, and ranks by that.

2 Uplift in five ideas

1. **The effect we want is a difference of two worlds.** For customer i , the uplift is $\tau_i = P(\text{churn} \mid \text{no offer}) - P(\text{churn} \mid \text{offer})$. Positive means the offer helps. We never see both worlds for one person.
2. **A randomized test makes groups comparable.** If a coin decides who gets the offer, treated and control customers are alike on average, so the difference in churn rates estimates the average effect. Before trusting it we check that the split worked (SRM) and that the groups look alike (balance).
3. **Uplift models estimate the effect per customer** from features, using both arms of the test: T-learner, X-learner, causal forest, and an IPW (transformed outcome) learner. Section 10 explains each.
4. **Evaluation is hard because the true effect per person is never observed.** On simulated data we know it and can measure error directly (PEHE). On real data we can only check whether high-ranked customers respond more (Qini) and how many extra customers a targeting rule keeps (policy value).
5. **Decisions use money, not probabilities.** Net value = $\hat{\tau} \times \text{customer value} - \text{offer cost}$. Only positive net value earns a contact.

3 System map

The project has two halves. A batch pipeline produces trusted numbers and a model. An agent uses them to answer questions and build offer lists that a reviewer checks.

```

Batch pipeline (weekly; Airflow or one command)
  generate -> validate -> analyze -> benchmark -> train -> gates -> register -> score -> agent
    |           |           |           |           |           |           |           |
  simulate   data    SRM, SMD,  30 draws:  fit    pass?    MLflow    score current
  50K A/B    contract ATE (Lin), PEHE, Qini, primary promote champion subscribers
  test                AIPW check policy value model or not alias

Agent (LangGraph, on request: CLI, API, or UI)
  router -> memory(load) -> sql -> ml -> rag -> policy -> reviewer -> save_offers -> report ->
    ↪ memory(save)

                                ^           |
                                +-- retry -+ (fixable issues, at most 2)

Questions about methods skip ml; questions about the test or one customer skip policy.

```

4 Repository map

Folder	Contents
app/core	Settings from YAML and environment variables
app/data	Simulation, SQLite store, loaders for public datasets
app/causal	Experiment checks, learners, metrics, bootstrap, benchmark, selector, policy, scorer
app/services	SQL access, knowledge retrieval, memory, MLflow registry
app/agents	LangGraph graph and the sub-agents
app/api, app/ui	FastAPI service and Streamlit dashboard
pipelines	Pipeline steps, quality gates, agent evaluation
scripts	Command-line helpers: ask the agent, roll back, real-data benchmark, Criteo sampling
configs	<code>base.yaml</code> (run settings), <code>sim.yaml</code> (simulation settings)
docs/knowledge	Short method cards the RAG agent searches
docs/manual	These manuals (LaTeX sources and PDFs)
infra	Dockerfile and Airflow DAG
evals	Golden cases for the agent evaluation
artifacts	Everything the pipeline writes (git-ignored except one summary file)

5 Results at a glance

All numbers come from one full run (`python -m pipelines.run_pipeline`, seed 0, 50,000 subscribers, 30 simulation draws). They are printed from `artifacts/reports/benchmark_summary.json` when the guide is built, so they always match the latest run.

Question	Answer
Was the A/B test valid?	Yes. SRM $p = 0.567$ (25,064 vs 24,936); largest $ SMD = 0.022$.
Did outreach reduce churn?	Lin-adjusted estimate 0.58 pp, 95% CI 0.05 to 1.11 pp (significant at 5%). The unadjusted difference was 0.76 pp (CI 0.20 to 1.32). The true simulated effect is 0.62 pp.
Does uplift targeting beat risk ranking?	In 30 simulated tests, contacting the top 20% by causal-forest uplift kept a median 19.1% more truly incremental customers than risk ranking (bootstrap 95% CI 9.2% to 33.0%; ratio of totals 19.3%; 48.5 vs 40.6 customers per 3,000 contacts; uplift won 26 of 30 draws).
Could one A/B readout see that?	No. On a 15,000-row holdout the observed difference was -2 customers (95% CI -100 to $+87$).
Do the models get individual effects right?	Only roughly. Predicting the same effect for everyone has PEHE 1.64 pp; only the causal forest does slightly better (1.61), the X-learner (1.78) and the others do worse. The gain comes mainly from ranking, not from precise individual estimates.
Which learner ranks best?	X-learner (true Qini 0.55 of the oracle), then the causal forest (0.47) and T-learner (0.46). The causal forest stays the primary model because it was named in the configuration before the benchmark ran.
Does a real offer list make money?	On a new simulated cohort, the 1,239 offers the agent proposed have a true net value of $+1,638$ dollars (predicted $+797$). In the benchmark, where models train on half the data, every learner's net-value policy lost money on average. Treat the cohort result as one example, not as typical.
Non-random assignment	Naive difference -11.81 pp; IPW -0.60 ; AIPW $+0.12$ pp (SE 0.35), whose CI covers the true 0.62.

What these results say

The simulated effect of outreach is small (0.62 pp on a 12% base) and varies little between customers (standard deviation 1.63 pp). In that setting a 25,000-per-arm test has only moderate power for the average effect, and a model can rank customers usefully while still being imprecise about each one's effect. Both are common in real retention programs.

Part II

The causal core

6 Configuration

All settings live in two YAML files. `configs/base.yaml` holds run settings: sample size, number of draws, coverage, paths, MLflow names. `configs/sim.yaml` holds the simulation parameters (the data-generating process). A few environment variables override YAML so CI and tests can run a smaller job without editing files.

`app/core/config.py`, lines 24–52

```

24 def _env_overrides(cfg: dict[str, Any]) -> dict[str, Any]:
25     """Apply UPLIFT_* environment variables on top of the YAML values.
26
27     Only a few keys are overridable; CI uses them to run a smaller job.
28     """
29     env = os.environ
30     if "UPLIFT_N" in env:
31         cfg["n_subscribers"] = int(env["UPLIFT_N"])
32     if "UPLIFT_SEED" in env:
33         cfg["seed"] = int(env["UPLIFT_SEED"])
34     if "UPLIFT_BOOTSTRAP" in env:
35         cfg["benchmark"]["bootstrap_reps"] = int(env["UPLIFT_BOOTSTRAP"])
36     if "UPLIFT_TREES" in env:
37         cfg["benchmark"]["forest_trees"] = int(env["UPLIFT_TREES"])
38     if "UPLIFT_DRAWS" in env:
39         cfg["benchmark"]["n_draws"] = int(env["UPLIFT_DRAWS"])
40     if "UPLIFT_ARTIFACTS" in env:
41         # Point every artifact path at another folder (used by tests).
42         base = Path(env["UPLIFT_ARTIFACTS"])
43         cfg["paths"]["artifacts"] = str(base)
44         cfg["paths"]["database"] = str(base / "uplift.db")
45         cfg["paths"]["report"] = str(base / "reports" / "latest_run.json")
46         cfg["paths"]["model"] = str(base / "models" / "uplift_scorer.joblib")
47         cfg["mlflow"]["tracking_uri"] = f"sqlite:///{(base / 'mlflow.db').as_posix()}"
48     if "AGENT_PROVIDER" in env:
49         cfg["agent"]["provider"] = env["AGENT_PROVIDER"]
50     if "MLFLOW_TRACKING_URI" in env:
51         cfg["mlflow"]["tracking_uri"] = env["MLFLOW_TRACKING_URI"]
52     return cfg

```

`get_settings` is cached, so every module sees the same settings in one process. Tests call `reset_settings` after changing environment variables. Relative paths are turned into absolute paths under the project root, so commands work from any folder.

7 Simulation

Real data never tells us the true effect for each customer, so we simulate subscribers where we do know it. That lets us measure how well each method recovers the truth.

7.1 Customers

Each subscriber has tenure, plan type (annual or monthly), sessions in the last 30 days, days since last login (linked to sessions), price sensitivity, payment failures, support tickets, monthly fee, and region. The distributions are plain guesses for a consumer subscription product.

7.2 Churn without the offer

Churn risk is a logistic function of a risk index (fewer sessions, longer since login, short tenure, monthly plan, failures, tickets, and price sensitivity all raise it) plus an unobserved noise term. The intercept is solved by bisection so that the average churn rate without the offer is exactly 12%. That sets the base rate only. It does not touch the effect.

app/data/simulate.py, lines 93–105

```

93 def solve_intercept(index: np.ndarray, target: float, noise_sd: float) -> float:
94     """Find the intercept so the mean churn rate without outreach equals target.
95
96     This sets the base rate only. The treatment effect is not calibrated.
97     """
98     lo, hi = -10.0, 5.0
99     for _ in range(60): # bisection; mean churn rises with the intercept
100         mid = (lo + hi) / 2
101         if expected_mu0(index, mid, noise_sd).mean() < target:
102             lo = mid
103         else:
104             hi = mid
105     return (lo + hi) / 2

```

7.3 The effect of the offer

The offer removes a share $r(x)$ of a customer's churn risk, so $\tau(x) = \mu_0(x) \cdot r(x)$. The share comes from three stories:

app/data/simulate.py, lines 108–128

```

108 def relative_effect(df: pd.DataFrame, sim: dict[str, Any]) -> np.ndarray:
109     """r(x): share of churn risk removed by outreach (negative = harm).
110
111     Persuadables: price-sensitive monthly customers who are drifting away
112     (last login 7-30 days ago). A discount reaches them in time.
113     Lost causes: no login for 45+ days. Outreach changes nothing.
114     Sleeping dogs: long-tenure annual customers who barely use the product.
115     A message reminds them they are paying, and some cancel.
116     """
117     days = df["days_since_login"].to_numpy(float)
118     # 0.3 when very active, 1.0 inside 7-30 days, falls to 0 at 45 days.
119     timing = np.where(days < 7, 0.3, np.where(days <= 30, 1.0, np.clip((45 - days) / 15, 0,
120     ↪ 1)))
121     persuadable = (
122         df["price_sensitivity"].to_numpy() * timing * (1 - df["plan_annual"].to_numpy())
123     )
124     sleeping_dog = (
125         df["plan_annual"].to_numpy()
126         * (df["tenure_months"].to_numpy() > 24)
127         * (df["sessions_30d"].to_numpy() <= 3)
128     )
129     return sim["persuadable_effect"] * persuadable - sim["sleeping_dog_effect"] * sleeping_dog

```

- **Persuadables:** price-sensitive monthly customers whose last login was 7 to 30 days ago. The discount reaches them while they are drifting and removes up to 35% of their churn risk.
- **Lost causes:** no login for 45 days or more. The effect is zero.
- **Sleeping dogs:** annual customers with more than 24 months of tenure who barely use the product. The message reminds them they are paying, and their churn goes up by up to 15%.

Because the risk includes unobserved noise, the true effect given the observed features averages over that noise. The code does this with Gauss–Hermite quadrature (`expected_mu0`) so `true_cate` is exactly $E[\tau \mid x]$, the quantity a model can learn.

7.4 Assignment, outcome, value

app/data/simulate.py, lines 131–177

```

131 def simulate(
132     n: int,
133     seed: int,
134     sim: dict[str, Any],
135     design: str = "rct",
136 ) -> pd.DataFrame:
137     """Return one row per subscriber with features, assignment, outcome, and oracle columns.
138
139     design="rct": each customer is assigned to outreach by an independent coin flip.
140     design="observational": customer-success reps contact risky customers more often,
141     so the naive treated-vs-control difference is biased.
142     """
143     rng = np.random.default_rng(seed)
144     df = simulate_covariates(n, rng)
145
146     index = risk_index(df)
147     noise_sd = sim["risk_noise_sd"]
148     intercept = solve_intercept(index, sim["base_churn_rate"], noise_sd)
149     r = relative_effect(df, sim)
150
151     # Individual churn probabilities include unobserved noise.
152     noise = rng.normal(0.0, noise_sd, size=n)
153     mu0_i = _sigmoid(intercept + index + noise)
154     mu1_i = np.clip(mu0_i * (1 - r), 0, 1)
155
156     # The CATE given observed x averages over the noise: tau(x) = E[mu0 | x] * r(x).
157     mu0_x = expected_mu0(index, intercept, noise_sd)
158
159     if design == "rct":
160         e = np.full(n, sim["treat_share"])
161     elif design == "observational":
162         lo, hi = sim["propensity_clip"]
163         e = np.clip(_sigmoid(sim["obs_intercept"] + sim["obs_risk_slope"] * _std(index)), lo,
164             ↪ hi)
165     else:
166         raise ValueError(f"unknown design: {design}")
167
168     treatment = rng.binomial(1, e)
169     churn = rng.binomial(1, np.where(treatment == 1, mu1_i, mu0_i))
170
171     df["treatment"] = treatment
172     df["churn_30d"] = churn
173     df["value"] = (sim["margin"] * df["monthly_fee"] * sim["value_months"]).round(2)
174     df["offer_cost"] = (sim["contact_cost"] + sim["discount_share"] *
175         ↪ df["monthly_fee"]).round(2)
176     df["true_mu0"] = mu0_x
177     df["true_cate"] = mu0_x * r
178     df["true_propensity"] = e
179     return df

```

In the randomized design every customer gets an independent coin flip with probability 0.5, so the arm sizes vary slightly and the SRM test has something to check. In the observational design, reps contact risky customers more often; that makes the naive comparison badly biased and gives the

IPW and AIPW estimators a real job.

Customer value is 70% margin on twelve monthly fees (about \$75 to \$151). Offer cost is \$1 contact plus 20% of one fee (about \$2.80 to \$4.60). An offer pays back only when the effect is above roughly 3.3 percentage points, which very few customers reach.

Columns that exist only because the data is simulated (`true_mu0`, `true_cate`, `true_propensity`) are kept out of every table the agent can read.

8 Data store

One SQLite file (`artifacts/uplift.db`) holds the experiment, the current subscribers, scores, pending offers, contact history, and agent memory. SQLite needs no server and ships with Python. Reads that come from the agent go through a read-only connection, so even a query that slipped past the SQL checks could not change data:

`app/data/store.py`, lines 53–56

```
53 def query(self, sql: str, params: tuple = ()) -> pd.DataFrame:
54     """Run a SELECT on a read-only connection. Writes fail at the driver level."""
55     with self.connect(read_only=True) as conn:
56         return pd.read_sql(sql, conn, params=params)
```

9 Experiment analysis

The order of checks matters. If the split is broken or the groups differ before treatment, no effect estimate can be trusted, so the pipeline stops there.

9.1 Sample ratio mismatch

`app/causal/experiment.py`, lines 18–33

```
18 def srm_test(n_treat: int, n_control: int, expected_share: float = 0.5) -> dict[str, float]:
19     """Sample ratio mismatch: chi-square test of arm sizes against the planned split.
20
21     A tiny p-value means assignment or logging is broken (e.g. a bug drops
22     some treated users). Industry practice alerts at  $p < 0.001$ .
23     """
24     n = n_treat + n_control
25     expected = np.array([n * expected_share, n * (1 - expected_share)])
26     chi2, p = stats.chisquare([n_treat, n_control], f_exp=expected)
27     return {
28         "n_treat": int(n_treat),
29         "n_control": int(n_control),
30         "treat_share": n_treat / n,
31         "chi2": float(chi2),
32         "p_value": float(p),
33     }
```

A chi-square test compares the arm sizes with the planned share (from `sim.yaml`, not hard-coded). The industry convention is to alert at $p < 0.001$; a mismatch usually means a logging or assignment bug.

9.2 Covariate balance

app/causal/experiment.py, lines 36–45

```

36 def standardized_mean_diff(x: pd.DataFrame, t: np.ndarray) -> pd.Series:
37     """SMD per column: (mean_T - mean_C) / sqrt((var_T + var_C) / 2).
38
39     Unlike a t-test, SMD does not shrink with sample size, so it measures
40     whether the gap is large enough to matter. |SMD| < 0.1 is the usual bar.
41     """
42     t = np.asarray(t).astype(bool)
43     xt, xc = x[t], x[~t]
44     pooled = np.sqrt((xt.var(ddof=1) + xc.var(ddof=1)) / 2).replace(0, np.nan)
45     return ((xt.mean() - xc.mean()) / pooled).fillna(0.0)

```

SMD divides the difference in means by the pooled standard deviation. Unlike a t-test it does not shrink with sample size, so it measures whether a gap is large enough to matter. All $|SMD| < 0.1$ passes.

9.3 Average effect

Two estimators are reported. The Lin (2013) regression adjustment was chosen *before* looking at results as the primary one. It regresses churn on treatment, centered covariates, and their interactions, with HC2 robust standard errors. In a randomized test it stays consistent and usually narrows the interval. The plain difference in means with a Neyman standard error is reported beside it.

app/causal/experiment.py, lines 69–88

```

69 def ate_lin_adjusted(
70     y: np.ndarray, t: np.ndarray, x: pd.DataFrame, alpha: float = 0.05
71 ) -> dict[str, float]:
72     """Lin (2013) regression adjustment:  $y \sim t + x_c + t * x_c$ , x centered, HC2 errors.
73
74     Using covariates that predict churn shrinks the standard error without
75     adding bias in a randomized test. The coefficient on t is the ATE.
76     """
77     t = np.asarray(t, float)
78     xc = (x - x.mean()).to_numpy(float)
79     design = np.column_stack([np.ones_like(t), t, xc, xc * t[:, None]])
80     fit = sm.OLS(np.asarray(y, float), design).fit(cov_type="HC2")
81     est, se = -fit.params[1], fit.bse[1] # negate: report churn reduction
82     z = stats.norm.ppf(1 - alpha / 2)
83     return {
84         "estimate": float(est),
85         "se": float(se),
86         "ci_low": float(est - z * se),
87         "ci_high": float(est + z * se),
88     }

```

Which estimate is the headline

The Lin estimate (0.58 pp, significant at 5%) is the headline because it was chosen in advance; the difference in means (0.76 pp) is reported beside it. Picking whichever number looks better after the fact would be cherry-picking. The two differ by less than one standard error. Power matters here: with 12% base churn and 25,000 per arm, the smallest effect the test detects with 80% power is about 0.8 pp, so an effect near 0.6 pp reaches significance in only about half of such tests.

9.4 When assignment is not random

On the observational variant the pipeline compares three estimates with the truth. Cross-fitted AIPW combines a propensity model and two outcome models; it stays unbiased if either part is right.

app/causal/experiment.py, lines 118–153

```

118 def aipw_ate(x: pd.DataFrame, t: np.ndarray, y: np.ndarray, folds: int = 5, seed: int = 0) ->
    ↪ dict:
119     """Cross-fitted AIPW (doubly robust) estimate of the churn reduction, with a standard
    ↪ error.
120
121     For each fold, the propensity model and the two outcome models are fit on
122     the other folds and used to score this fold, so no unit is scored by a
123     model that saw it. The estimate is the mean of the DR scores; its SE is
124     their standard deviation / sqrt(n).
125
126     The propensity model is a plain logistic regression, chosen for how well
127     it predicts treatment, not borrowed from the heavily regularized CATE
128     settings. Those settings pull  $e(x)$  toward 0.5, under-weight the rare
129     high-risk controls, and leave bias (an earlier version missed by ~4 SE).
130     Whether the weights balance the arms is checked afterwards (weighted SMD).
131     """
132     from lightgbm import LGBMClassifier
133     from sklearn.model_selection import StratifiedKFold
134
135     from app.causal.learners import LGBM_PARAMS, Propensity
136     from app.causal.metrics import dr_pseudo_outcome
137
138     t, y = np.asarray(t), np.asarray(y, float)
139     n = len(t)
140     e, m0, m1 = np.zeros(n), np.zeros(n), np.zeros(n)
141     for k, (fit_idx, score_idx) in enumerate(
142         StratifiedKFold(folds, shuffle=True, random_state=seed).split(x, t)
143     ):
144         xf, tf, yf = x.iloc[fit_idx], t[fit_idx], y[fit_idx]
145         xs = x.iloc[score_idx]
146         e[score_idx] = Propensity(known=None).fit(xf, tf).predict(xs) # clipped to [0.05,
    ↪ 0.95]
147         c0 = LGBMClassifier(random_state=seed + k, **LGBM_PARAMS).fit(xf[tf == 0], yf[tf == 0])
148         c1 = LGBMClassifier(random_state=seed + k, **LGBM_PARAMS).fit(xf[tf == 1], yf[tf == 1])
149         m0[score_idx] = c0.predict_proba(xs)[:, 1]
150         m1[score_idx] = c1.predict_proba(xs)[:, 1]
151     phi = dr_pseudo_outcome(y, t, m0, m1, e)
152     est, se = float(phi.mean()), float(phi.std(ddof=1) / np.sqrt(n))
153     return {"estimate": est, "se": se, "ci_low": est - 1.96 * se, "ci_high": est + 1.96 * se,
    ↪ "propensity": e}

```

An earlier version reused the heavily regularized LightGBM settings from the uplift learners for the propensity model. Those settings pull $\hat{e}(x)$ toward 0.5, so the rare high-risk control customers got about half their correct weight, and AIPW missed the truth by about four standard errors. The weighted balance check showed it (max SMD 0.16 after weighting). Fitting the propensity with plain logistic regression, chosen for how well it predicts treatment, brought the weighted max SMD to 0.038 and the estimate to +0.12 pp (SE 0.35), whose CI covers the true 0.62 pp. A slight negative lean remains because the logistic model is not exactly the true assignment rule; it is reported as it is.

10 Uplift learners

Every learner has the same interface: `fit(x, t, y)` with $y = \text{churn}$, and `predict(x)` returning the predicted churn *reduction*. A larger score always means “contact this customer”.

10.1 Baselines

Churn risk is trained on the control arm only, so it predicts churn without an offer. It is the status quo the uplift models must beat.

k-means segments cluster customers on standardized features and give everyone in a cluster that cluster’s A/B difference:

app/causal/learners.py, lines 91–117

```

91 class KMeansUplift:
92     """Segment baseline: cluster customers, then use each cluster's A/B difference.
93
94     This is the "segment-level" approach many teams start with. Every customer
95     in a cluster gets the same effect estimate.
96     """
97
98     name = "kmeans"
99
100    def __init__(self, k: int = 8, seed: int = 0):
101        self.k, self.seed = k, seed
102
103    def fit(self, x: pd.DataFrame, t: np.ndarray, y: np.ndarray) -> KMeansUplift:
104        t, y = np.asarray(t), np.asarray(y, float)
105        self.scaler = StandardScaler().fit(x)
106        self.km = KMeans(n_clusters=self.k, n_init=10, random_state=self.seed)
107        labels = self.km.fit_predict(self.scaler.transform(x))
108        self.cluster_tau = np.zeros(self.k)
109        for c in range(self.k):
110            in_c = labels == c
111            treated, control = in_c & (t == 1), in_c & (t == 0)
112            if treated.any() and control.any():
113                self.cluster_tau[c] = y[control].mean() - y[treated].mean()
114        return self
115
116    def predict(self, x: pd.DataFrame) -> np.ndarray:
117        return self.cluster_tau[self.km.predict(self.scaler.transform(x))]

```

Constant ATE gives every customer the same effect. It cannot rank anyone, but it is the right yardstick for PEHE: a learner that cannot beat it has not found real differences between customers.

10.2 T-learner

Two classifiers, one per arm; the uplift is the difference of their predictions. Simple, but each model sees half the data and their errors do not cancel.

10.3 X-learner

app/causal/learners.py, lines 142–174

```

142 class XLearner:
143     """Kuenzel et al. (2019). Impute each unit's effect, then model the effect directly.
144

```

```

145 Step 1: fit mu0, mu1 as in the T-learner.
146 Step 2: imputed effects on churn,
147         treated units: D1 = y - mu0(x); control units: D0 = mu1(x) - y.
148 Step 3: regress D1 and D0 on x to get tau1(x), tau0(x).
149 Step 4: blend, tau(x) = e(x) * tau0(x) + (1 - e(x)) * tau1(x).
150 Helps most when one arm is much larger than the other.
151 """
152
153 name = "x_learner"
154
155 def __init__(self, seed: int = 0, propensity: Propensity | None = None, params: dict |
156 ↪ None = None):
157     self.seed, self.params = seed, params
158     self.propensity = propensity or Propensity(known=0.5)
159
160 def fit(self, x: pd.DataFrame, t: np.ndarray, y: np.ndarray) -> XLearner:
161     t, y = np.asarray(t), np.asarray(y, float)
162     x0, x1 = x[t == 0], x[t == 1]
163     m0 = _clf(self.seed, self.params).fit(x0, y[t == 0])
164     m1 = _clf(self.seed + 1, self.params).fit(x1, y[t == 1])
165     d1 = y[t == 1] - m0.predict_proba(x1)[0, 1]
166     d0 = m1.predict_proba(x0)[0, 1] - y[t == 0]
167     self.tau1 = _reg(self.seed + 2, self.params).fit(x1, d1)
168     self.tau0 = _reg(self.seed + 3, self.params).fit(x0, d0)
169     self.propensity.fit(x, t)
170     return self
171
172 def predict(self, x: pd.DataFrame) -> np.ndarray:
173     e = self.propensity.predict(x)
174     effect_on_churn = e * self.tau0.predict(x) + (1 - e) * self.tau1.predict(x)
175     return -effect_on_churn

```

The X-learner (Künzel et al. 2019) first fits the two outcome models, then imputes an effect for every customer from the *other* arm's model, regresses those imputed effects on features, and blends the two effect models with the propensity. It helps most when arms are unbalanced. In this project it ranked best (Section 5).

10.4 Causal forest

app/causal/learners.py, lines 177–208

```

177 class CausalForest:
178     """Generalized random forest (Athey, Tibshirani, Wager 2019) via econml CausalForestDML.
179
180     First removes the part of y and t that x predicts (double ML), then grows
181     trees that split on where the effect differs. Honest splitting gives
182     valid confidence intervals.
183     """
184
185     name = "causal_forest"
186
187     def __init__(self, seed: int = 0, n_estimators: int = 300, known_propensity: bool = True):
188         from econml.dml import CausalForestDML
189         from sklearn.dummy import DummyClassifier
190
191         # In an RCT the propensity is a constant; the class prior estimates it exactly.
192         model_t = DummyClassifier(strategy="prior") if known_propensity else
193         ↪ LogisticRegression(max_iter=1000)
194         self.est = CausalForestDML(
195             model_y=_reg(seed),
196             model_t=model_t,
197             discrete_treatment=True,

```



```

197         n_estimators=max(4, n_estimators // 4 * 4), # econml grows trees in groups of 4
198         min_samples_leaf=100,
199         cv=2,
200         random_state=seed,
201     )
202
203     def fit(self, x: pd.DataFrame, t: np.ndarray, y: np.ndarray) -> CausalForest:
204         self.est.fit(np.asarray(y, float), np.asarray(t), X=x.to_numpy(float))
205         return self
206
207     def predict(self, x: pd.DataFrame) -> np.ndarray:
208         return -self.est.effect(x.to_numpy(float)).ravel()

```

econml's `CausalForestDML` removes what the features predict about churn and about treatment (double machine learning), then grows honest trees that split where the effect differs. In a randomized test the propensity is a known constant, so a `DummyClassifier` with the class prior is the exact model for it.

10.5 IPW learner

app/causal/learners.py, lines 211–233

```

211 class IPWLearner:
212     """Transformed-outcome learner (Athey and Imbens 2016), an IPW method for CATE.
213
214      $Y^* = y * (t - e) / (e * (1 - e))$  has  $E[Y^* | x] = \mu_1(x) - \mu_0(x)$ , so any
215     regression of  $Y^*$  on  $x$  estimates the effect. Unbiased, but  $Y^*$  is very noisy
216     (values like +2 or -2 for every churner), so it needs a lot of data.
217     """
218
219     name = "ipw"
220
221     def __init__(self, seed: int = 0, propensity: Propensity | None = None, params: dict |
222     ↪ None = None):
223         self.seed, self.params = seed, params
224         self.propensity = propensity or Propensity(known=0.5)
225
226     def fit(self, x: pd.DataFrame, t: np.ndarray, y: np.ndarray) -> IPWLearner:
227         t, y = np.asarray(t, float), np.asarray(y, float)
228         e = self.propensity.fit(x, t).predict(x)
229         y_star = y * (t - e) / (e * (1 - e))
230         self.model = _reg(self.seed, self.params).fit(x, y_star)
231         return self
232
233     def predict(self, x: pd.DataFrame) -> np.ndarray:
234         return -self.model.predict(x)

```

The transformed outcome $Y^* = y(t - e)/(e(1 - e))$ has expected value equal to the effect, so any regressor fit to Y^* estimates uplift. It is unbiased but very noisy, since every churner becomes ± 2 when $e = 0.5$.

10.6 Regularization

The true effect varies by only about 1.5 pp between customers. With default tree settings the first version of the T-learner, X-learner, and IPW had PEHE of 4.6 to 7.6 pp, far worse than predicting zero. The fix was principled rather than tuned to a number: small, shallow trees, and a four-point grid chosen per learner by the doubly robust loss on validation data (Section 13), which needs no true effect.

app/causal/learners.py, lines 30–34

```

30 GRID = [
31     {"num_leaves": leaves, "min_child_samples": child}
32     for leaves in (4, 8)
33     for child in (500, 1000)
34 ]

```

11 Metrics

11.1 PEHE

Root mean squared error between estimated and true effect. Only possible on simulated data.

11.2 Qini

Sort customers by score. For the top k , the Qini curve counts extra retained customers: treated retained minus control retained, with the control count rescaled to the treated group size.

app/causal/metrics.py, lines 80–98

```

80 def qini_curve(score, retained, t, n_points: int = 100) -> tuple[np.ndarray, np.ndarray]:
81     """Observed incremental retained customers when contacting the top phi share.
82
83      $Q(k) = R_T(k) - R_C(k) * N_T(k) / N_C(k)$ 
84     R_T, R_C: retained counts in the treated / control part of the top k.
85     N_T, N_C: treated / control counts in the top k.
86     The control count is rescaled to the treated group size (Radcliffe 2007).
87     """
88     order, group = _order_and_groups(score)
89     y = np.asarray(retained, float)[order]
90     tt = np.asarray(t)[order]
91     rt = np.cumsum(y * (tt == 1))
92     rc = np.cumsum(y * (tt == 0))
93     nt = np.cumsum(tt == 1)
94     nc = np.cumsum(tt == 0)
95     q = rt - np.divide(rc * nt, nc, out=np.zeros_like(rc), where=nc > 0)
96     q = _interpolate_ties(np.r_[0.0, q], group)
97     phi, k = _grid(len(y), n_points)
98     return np.r_[0.0, phi], np.r_[0.0, q[k]]

```

The Qini coefficient is the average gap between that curve and the straight line of random targeting, divided by the number of treated customers.

11.3 True Qini

With simulated data the same idea can use the true effects instead of noisy outcomes: $G(k)$ is the sum of true τ over the top k . The normalized version divides by the value for a perfect ranking, so 1 is perfect, 0 is random, and it can go negative.

app/causal/metrics.py, lines 63–71

```

63 def true_qini(score, tau, n_points: int = 100) -> float:
64     """Noise-free Qini:  $Q^* = (1 / (M n)) * \sum_m [G(k_m) - \phi_m * G(n)]$ .
65
66     Random targeting gives 0. Sorting by the true tau gives the maximum.
67     """
68     g = expected_gain_curve(score, tau)

```

```

69     n = len(g) - 1
70     phi, k = _grid(n, n_points)
71     return float(np.mean(g[k] - phi * g[n]) / n)

```

11.4 Ties

k-means gives only eight distinct scores, so large groups of customers are tied. Breaking ties in row order would make results depend on how the file is sorted. The curves instead take the expected value under random tie-breaking: inside each tie group the cumulative curve is a straight line between the group's start and end.

app/causal/metrics.py, lines 36–48

```

36 def _interpolate_ties(cum: np.ndarray, group: np.ndarray) -> np.ndarray:
37     """Replace a cumulative curve inside each tie group by a straight line
38     between the group's start and end. This is its expected value under
39     random tie-breaking. cum has length n+1 with cum[0] = 0."""
40     n = len(group)
41     ends = np.flatnonzero(np.r_[group[1:] != group[:-1], True]) + 1 # exclusive end per group
42     starts = np.r_[0, ends[:-1]]
43     pos = np.arange(1, n + 1)
44     g_start, g_end = starts[group], ends[group]
45     frac = (pos - g_start) / (g_end - g_start)
46     out = cum.copy()
47     out[1:] = cum[g_start] + frac * (cum[g_end] - cum[g_start])
48     return out

```

11.5 Doubly robust score

app/causal/metrics.py, lines 156–164

```

156 def dr_pseudo_outcome(y, t, m0, m1, e) -> np.ndarray:
157     """Doubly robust (AIPW) score for the churn REDUCTION of each unit.
158
159     phi = (m0 - m1) + (1-t)(y - m0)/(1-e) - t(y - m1)/e
160     E[phi | x] equals the true reduction if either the outcome models or the
161     propensity are right; in an RCT e is known, so phi is unbiased.
162     """
163     y, t = np.asarray(y, float), np.asarray(t, float)
164     return (m0 - m1) + (1 - t) * (y - m0) / (1 - e) - t * (y - m1) / e

```

$E[\varphi | x]$ equals the true effect when either the outcome models or the propensity is right; in a randomized test the propensity is known, so it is always right. The DR loss $\text{mean}((\varphi - \hat{\tau})^2)$ equals PEHE² plus a constant that is the same for every model, so it ranks models the way PEHE would, without knowing τ .

12 Bootstrap

The bootstrap answers one narrow question: if we had drawn a different test set of the same size, how much would the observed comparison move? Models stay frozen; only test rows are resampled, separately inside each arm so arm sizes stay fixed.

app/causal/bootstrap.py, lines 13–27

```

13 def stratified_weights(t: np.ndarray, reps: int, seed: int) -> np.ndarray:
14     """Return a (reps, n) matrix of resample counts.

```

```

15
16     Rows are drawn with replacement separately inside the treated and control
17     arms, so each replicate keeps the same arm sizes as the real test set.
18     """
19     rng = np.random.default_rng(seed)
20     t = np.asarray(t)
21     w = np.zeros((reps, len(t)), dtype=np.int32)
22     for arm in (0, 1):
23         idx = np.flatnonzero(t == arm)
24         draws = rng.integers(0, len(idx), size=(reps, len(idx)))
25         for b in range(reps):
26             np.add.at(w[b], idx[draws[b]], 1)
27     return w

```

The result is a matrix of resample counts, so each replicate's incremental retention is a matrix product rather than a Python loop:

app/causal/bootstrap.py, lines 30–39

```

30 def incremental_retained(w: np.ndarray, mask: np.ndarray, retained: np.ndarray, t: np.ndarray)
   ↪ -> np.ndarray:
31     """Per replicate: (treated retention - control retention) in the selected group * group
   ↪ size."""
32     y = np.asarray(retained, float)
33     sel_t = (mask & (t == 1)).astype(float)
34     sel_c = (mask & (t == 0)).astype(float)
35     n_t = w @ sel_t
36     n_c = w @ sel_c
37     rate_t = (w @ (sel_t * y)) / np.maximum(n_t, 1)
38     rate_c = (w @ (sel_c * y)) / np.maximum(n_c, 1)
39     return (rate_t - rate_c) * (n_t + n_c)

```

A mistake that was fixed

The first version pooled bootstrap replicates from five overlapping train/test splits and reported the median of the ratio $(I_{\text{uplift}} - I_{\text{risk}})/|I_{\text{risk}}|$. That pooled set does not estimate anything well defined, and the ratio explodes when I_{risk} is near zero (the interval ran from -354% to $+1382\%$). The current version reports the absolute gain for the observed readout and takes the headline from independent simulation draws.

13 Benchmark

For each of 30 draws the benchmark simulates a fresh set of 50,000 subscribers, splits it 50/20/30 into train, validation, and test, fits all models on train, and scores the test set.

app/causal/benchmark.py, lines 61–78

```

61 def fit_models(x_tr, t_tr, y_tr, x_va, phi_va, seed: int, cfg: dict[str, Any], treat_share:
   ↪ float):
62     """Fit baselines and uplift learners; grid-search the tunable ones by validation DR
   ↪ loss."""
63     models, info = {}, {}
64     for name in BASELINES + UPLIFT_LEARNERS:
65         start = time.perf_counter()
66         candidates = GRID if name in TUNABLE else [None]
67         best = None
68         for params in candidates:
69             model = make_learner(
70                 name, seed=seed, treat_share=treat_share, forest_trees=cfg["forest_trees"],
71                 kmeans_k=cfg["kmeans_k"], params=params,

```

```

72         ).fit(x_tr, t_tr, y_tr)
73         loss = M.dr_loss(model.predict(x_va), phi_va) if name != "churn_risk" else None
74         if best is None or (loss is not None and loss < best[1]):
75             best = (model, loss, params)
76         models[name] = best[0]
77         info[name] = {"dr_loss": best[1], "params": best[2], "fit_seconds":
78             ↳ round(time.perf_counter() - start, 2)}
79     return models, info

```

The primary model (causal forest) is named in `base.yaml` before any run, so the headline involves no data-driven choice. A secondary choice by validation DR loss, with a one-standard-error rule that falls back to the primary, is reported beside it (counts over draws: causal forest 23, x learner 4, ipw 3).

The headline compares true policy value at 20% coverage across draws. Because the draws are independent, their spread is a valid measure of uncertainty that includes training and test noise. For each draw the relative gain is the primary model's true policy value divided by the risk ranking's, minus one. The headline is the median of these per-draw gains with a bootstrap interval from resampling draws. The ratio is stable because the risk ranking's true value is always well above zero. A ratio of totals is reported beside it:

app/causal/benchmark.py, lines 172–178

```

172 def median_with_ci(values, reps: int = 2000, seed: int = 0) -> dict[str, float]:
173     """Median of per-draw values with a percentile interval from resampling draws."""
174     v = np.asarray(values, float)
175     rng = np.random.default_rng(seed)
176     boot = np.median(v[rng.integers(0, len(v), size=(reps, len(v)))], axis=1)
177     lo, hi = np.percentile(boot, [2.5, 97.5])
178     return {"median": float(np.median(v)), "ci_low": float(lo), "ci_high": float(hi),
179         ↳ "n_draws": int(len(v))}

```

app/causal/benchmark.py, lines 161–169

```

161 def ratio_of_totals(model_vals, risk_vals, reps: int = 2000, seed: int = 0) -> dict[str,
162     ↳ float]:
163     """sum(model) / sum(risk) - 1 across draws, with a percentile interval from resampling
164     ↳ draws."""
165     a, b = np.asarray(model_vals, float), np.asarray(risk_vals, float)
166     point = a.sum() / b.sum() - 1
167     rng = np.random.default_rng(seed)
168     idx = rng.integers(0, len(a), size=(reps, len(a)))
169     boot = a[idx].sum(1) / b[idx].sum(1) - 1
170     lo, hi = np.percentile(boot, [2.5, 97.5])
171     return {"estimate": float(point), "ci_low": float(lo), "ci_high": float(hi), "n_draws":
172         ↳ int(len(a))}

```

14 Estimator selector

The same workflow should behave sensibly on data that looks different. `profile_data` records the facts that matter; `select` applies written rules, each with its reason, so the agent can explain the choice and the reviewer can check it.

app/causal/selector.py, lines 69–109

```

69 def select(p: DataProfile) -> Selection:
70     """Rule table. Rules are checked in order; later rules adjust earlier choices."""
71     s = Selection(estimators=[], policy="net_value_top_k", propensity="known")
72

```

Model	PEHE (pp)	True Qini (norm.)	Policy value at 20%	Net-value policy (\$)
Churn risk	–	0.39	40.6	–
k-means	1.88	0.13	22.5	–252
Constant ATE	1.64	0.00	18.2	0
T-learner	2.51	0.46	50.1	–2,127
X-learner	1.78	0.55	53.2	–620
Causal forest	1.61	0.47	48.5	–154
IPW	2.35	0.43	45.5	–2,131
Oracle (true τ)	0	1.00	85.6	1,801
Random	–	0.00	18.4	–

Table 1: Means over 30 draws. Policy value is extra customers kept per 3,000 contacts. The net-value policy contacts only customers with positive predicted net value and reports their true net value.

```

73     if p.design == "observational":
74         s.estimators = ["causal_forest", "ipw", "x_learner"]
75         s.propensity = "estimated"
76         s.reasons.append("Not randomized: treatment depends on x, so we model the propensity "
77                         "and use estimators that adjust for it. A plain T-learner is not
78                         ↪ allowed.")
79         s.warnings.append("Effects rely on no unmeasured confounding; check propensity
80                         ↪ overlap.")
81     elif not 0.3 <= p.treat_share <= 0.7:
82         s.estimators = ["x_learner", "causal_forest", "ipw", "t_learner"]
83         s.reasons.append(f"Arms are unbalanced (treated share {p.treat_share:.2f}); the
84                         ↪ X-learner "
85                         "borrows strength from the large arm to model the small one.")
86     else:
87         s.estimators = ["causal_forest", "x_learner", "t_learner", "ipw"]
88         s.reasons.append("Balanced randomized test: all learners are valid; propensity is
89                         ↪ known.")
90
91     if p.positive_rate is not None and p.positive_rate < RARE_OUTCOME:
92         s.estimators = [e for e in s.estimators if e != "t_learner"]
93         s.warnings.append(f"Rare outcome ({p.positive_rate:.1%}): effect estimates are noisy; "
94                         "expect wide intervals and prefer pooled learners.")
95         s.reasons.append("Rare outcome: dropped the T-learner, whose two separate models each
96                         ↪ see few events.")
97
98     if p.n > LARGE_N:
99         s.estimators = [e for e in s.estimators if e in ("t_learner", "x_learner", "ipw")] or
100         ↪ ["t_learner"]
101         s.reasons.append(f"N = {p.n:,} is large: skip the causal forest for run time; fit on a
102                         ↪ subsample if needed.")
103
104     if p.n_arms > 2:
105         s.policy = "best_arm_net_value"
106         s.reasons.append(f"{p.n_arms} arms: fit each treatment arm against control and give
107                         ↪ each "
108                         "customer the arm with the highest net value, or no contact.")
109
110     if p.outcome_type == "continuous":
111         s.reasons.append("Continuous outcome: learners use regressors and the effect is in
112                         ↪ outcome units.")
113
114     if not p.has_value_cost:
115         s.policy = "tau_top_k" if p.n_arms <= 2 else "best_arm_tau"
116         s.reasons.append("No value or cost columns: rank by estimated effect. Never rank by
117                         ↪ risk.")
118

```

```
109     return s
```

Whether assignment was randomized cannot be read off the data, so `design` is passed in by whoever owns the data. Section 16 shows the rules on three public datasets.

15 Targeting policy

app/causal/policy.py, lines 23–42

```
23 def rank_by_net_value(
24     scored: pd.DataFrame,
25     budget: int,
26     exclude_ids: set[int] | None = None,
27     min_net_value: float = 0.0,
28 ) -> pd.DataFrame:
29     """Pick up to `budget` customers with the highest positive net value.
30
31     scored needs: customer_id, tau_hat, value, offer_cost.
32     exclude_ids: customers contacted recently (cooldown), from memory.
33     """
34     df = scored.copy()
35     df["net_value"] = net_value(df["tau_hat"], df["value"], df["offer_cost"])
36     if exclude_ids:
37         df = df[~df["customer_id"].isin(exclude_ids)]
38     df = df[(df["tau_hat"] > 0) & (df["net_value"] > min_net_value)]
39     df = df.sort_values("net_value", ascending=False).head(budget).reset_index(drop=True)
40     df["rank"] = np.arange(1, len(df) + 1)
41     df["offer"] = OFFER_NAME
42     return df
```

Customers with $\hat{\tau} \leq 0$ never get an offer, however risky they are. When fewer customers have positive net value than the budget allows, fewer offers go out; spending the whole budget is not a goal.

16 Real datasets

Three public uplift datasets exercise the selector. They are not in the repository; the Operation Manual says how to download them. Each loader maps its file to the same format, with the outcome we want more of called good (retained, visited).

scripts/run_real_benchmarks.py, lines 43–57

```
43 def evaluate(ds, estimators: list[str], seed: int = 0, trees: int = 200) -> dict:
44     """Fit each allowed learner on 70%, score Qini on 30%. Learners model bad = 1 - good."""
45     tr, te = train_test_split(np.arange(len(ds.t)), train_size=0.7, stratify=ds.t,
46                               ↪ random_state=seed)
47     share = float(ds.t[tr].mean()) # known assignment share of this randomized test
48     bad = 1 - ds.good
49     rows = {}
50     for name in ["churn_risk"] + estimators:
51         model = make_learner(name, seed=seed, treat_share=share, forest_trees=trees)
52         model.fit(ds.x.iloc[tr], ds.t[tr], bad[tr])
53         # churn_risk scores P(bad outcome without treatment): the usual "highest risk first"
54         ↪ list.
55         score = model.predict(ds.x.iloc[te])
56         rows[name] = qini_with_ci(score, ds.good[te], ds.t[te])
57     rng = np.random.default_rng(seed)
58     rows["random"] = qini_with_ci(rng.random(len(te)), ds.good[te], ds.t[te])
```

```
57     return {"n_train": len(tr), "n_test": len(te), "treat_share": share, "qini": rows}
```


Dataset	What the selector chose and why	Held-out Qini (95% CI)
Orange telecom churn (Verhelst et al. 2023), 11,896 rows, 76% treated, 3.4% churn	X-learner, causal forest, IPW; T-learner in-dropper for the rare outcome; rank by effect (no value data)	No model beats random; every learner in-dropper for the rare outcome; 0
Hillstrom e-mail (2008), 64,000 rows, 3 arms	All four learners, one model per arm against control	Womens e-mail: causal forest +0.0072 (+0.0041, +0.0101), X-learner +0.0072, T-learner +0.0066. Mens e-mail: none beats random
Criteo ads (Diemert et al. 2018), 70,086-row sample of 13.98M, 85% treated	Size of the full table	X-learner +0.0032 (−0.0001, +0.0062), bor-

17 Scorer bundle

Serving and the agent never need to know which learner is inside the model. They call one method:

app/causal/scorer.py, lines 31–39

```

31 def score(self, customers: pd.DataFrame) -> pd.DataFrame:
32     """Add tau_hat (churn reduction), churn_risk, and net_value columns."""
33     x = feature_matrix(customers)[self.feature_columns]
34     out = customers.copy()
35     out["tau_hat"] = self.uplift_model.predict(x)
36     out["churn_risk"] = self.risk_model.predict(x)
37     out["net_value"] = out["tau_hat"] * out["value"] - out["offer_cost"]
38     out["model_version"] = self.version
39     return out

```

The bundle is saved with joblib and logged to MLflow together with hashes of the training data and the configuration.

Part III

The agent system

18 Agent state

LangGraph passes one dictionary from node to node. Each node returns only the keys it changes. Services (database, SQL, knowledge base, memory, model) are built once per run and injected, so tests can swap in a copy of the database.

app/agents/state.py, lines 50–59

```

50 @dataclass
51 class Deps:
52     """Services injected into every node, built once per run."""
53
54     settings: dict[str, Any]
55     store: Store
56     sql: SQLService
57     kb: KnowledgeBase
58     memory: MemoryService
59     scorer: UpliftScorer | None

```

19 Agent graph

app/agents/graph.py, lines 73–105

```

73 def build_graph(deps: Deps):
74     g = StateGraph(AgentState)
75     nodes = {
76         "router": route,
77         "memory_load": load_memory,
78         "sql": run_sql_agent,

```

```

79     "ml": run_ml_agent,
80     "rag": run_rag_agent,
81     "policy": run_policy_agent,
82     "reviewer": run_reviewer,
83     "save_offers": save_offers,
84     "report": run_report,
85     "memory_save": save_memory,
86 }
87 for name, fn in nodes.items():
88     g.add_node(name, partial(fn, deps=deps))
89
90 g.add_edge(START, "router")
91 g.add_edge("router", "memory_load")
92 g.add_edge("memory_load", "sql")
93 g.add_conditional_edges("sql", after_sql, {"rag": "rag", "ml": "ml"})
94 g.add_edge("ml", "rag")
95 g.add_conditional_edges("rag", after_rag, {"policy": "policy", "report": "report"})
96 g.add_edge("policy", "reviewer")
97 max_retries = deps.settings["agent"]["max_review_retries"]
98 g.add_conditional_edges(
99     "reviewer", partial(after_review, max_retries=max_retries),
100     {"save_offers": "save_offers", "report": "report", "policy": "policy"},
101 )
102 g.add_edge("save_offers", "report")
103 g.add_edge("report", "memory_save")
104 g.add_edge("memory_save", END)
105 return g.compile()

```

The conditional edges encode the routing. A method question goes straight from SQL to retrieval to the report. A campaign request goes through policy and review. After review, a pass saves the list as pending; fixable problems send the list back to the policy agent (at most twice); blocking problems end the run with an explanation.

app/agents/graph.py, lines 64–70

```

64 def after_review(state: AgentState, max_retries: int) -> str:
65     review = state["review"]
66     if review["passed"]:
67         return "save_offers"
68     if review["blocking"] or state.get("retries", 0) > max_retries:
69         return "report"
70     return "policy"

```

20 Memory agent

At the start of a run the memory agent loads recent runs, reviewer feedback, and the cooldown list (customers contacted in the last 30 days). Reviewer exclusions from earlier campaigns become exclusions for this one. At the end it saves a short summary of the run.

app/agents/memory_agent.py, lines 11–24

```

11 def load_memory(state: AgentState, deps: Deps) -> AgentState:
12     days = deps.settings["agent"]["contact_cooldown_days"]
13     feedback = deps.memory.recall("feedback", limit=5)
14     # Reviewer feedback can carry customer exclusions for future campaigns.
15     excluded = {int(i) for f in feedback for i in f["content"].get("exclude_ids", [])}
16     return {
17         "memory": {
18             "recent_runs": deps.memory.recall("run", limit=3),
19             "feedback": feedback,

```

```

20         "cooldown_ids": sorted(deps.memory.recently_contacted(days)),
21         "excluded_ids": sorted(excluded),
22     },
23     "trace": state.get("trace", []) + ["memory_agent: loaded runs, feedback, cooldown
↪ list"],
24 }

```

21 SQL agent

app/agents/sql_agent.py, lines 18–27

```

18 def run_sql_agent(state: AgentState, deps: Deps) -> AgentState:
19     results, notes = {}, []
20     for name in QUERIES_BY_INTENT.get(state["intent"], []):
21         params = (state.get("customer_id"),) if name == "customer" else ()
22         try:
23             results[name] = deps.sql.run_template(name, params).to_dict("records")
24         except Exception as exc: # missing table before the first pipeline run
25             notes.append(f"{name} failed: {exc}")
26     trace = f"sql_agent: ran {'', '.join(results) or 'no queries'}" + (f" ({'; '.join(notes)})"
↪ if notes else "")
27     return {"sql": results, "trace": state.get("trace", []) + [trace]}

```

Each intent maps to named queries. Parameters are bound by the driver, never pasted into SQL text.

22 ML agent

app/agents/ml_agent.py, lines 25–61

```

25 def run_ml_agent(state: AgentState, deps: Deps) -> AgentState:
26     trace = list(state.get("trace", []))
27     exp_df = deps.store.read_table("experiment")
28
29     experiment = _report(deps, "experiment.json")
30     if experiment is None:
31         experiment = analyze_experiment(exp_df, feature_matrix(exp_df),
↪ expected_share=deps.settings["sim"]["treat_share"])
32         trace.append("ml_agent: no saved experiment report; analyzed the table directly")
33
34     # The design (randomized or not) comes from the data owner's metadata.
35     profile = profile_data(exp_df, "treatment", "churn_30d", design="rct", value_col="value",
↪ cost_col="offer_cost")
36     selection = select(profile)
37     trace.append("ml_agent: " + describe(profile, selection).splitlines()[1])
38
39     model = {"method": None, "version": None}
40     if deps.scorer is not None:
41         model = {"method": deps.scorer.method, "version": deps.scorer.version, "metrics":
↪ deps.scorer.metrics}
42         bench = _report(deps, "benchmark.json")
43         if bench:
44             model["headline"] = bench["headline"]
45             model["observed_readout"] = bench["observed_readout_draw0"]
46     else:
47         trace.append("ml_agent: no trained model found; run the pipeline first")
48
49     return {

```

```

50     "experiment": {
51         "valid": experiment["valid"],
52         "srm_p": experiment["srm"]["p_value"],
53         "max_abs_smd": experiment["balance"]["max_abs_smd"],
54         "ate_lin": experiment["ate_lin"],
55         "ate_diff_in_means": experiment["ate_diff_in_means"],
56     },
57     "profile": profile.__dict__,
58     "selection": selection.to_dict(),
59     "model": model,
60     "trace": trace,
61 }

```

The ML agent does not train models during a request; training belongs to the scheduled pipeline. It reads the experiment report, profiles the data, runs the selector, and attaches the champion model's version and benchmark headline.

23 RAG agent

app/agents/rag_agent.py, lines 20–31

```

20 def run_rag_agent(state: AgentState, deps: Deps) -> AgentState:
21     selection = state.get("selection") or {}
22     query = " ".join(
23         [state.get("question", ""), EXTRA_QUERY.get(state["intent"], "")]
24         + selection.get("estimators", [])[:1]
25         + selection.get("warnings", [])
26     )
27     hits = deps.kb.search(query, k=3)
28     return {
29         "knowledge": [{"citation": h.citation(), "text": h.text, "score": round(h.score, 3)}
30         ↪ for h in hits],
31         "trace": state.get("trace", []) + [f"rag_agent: {len(hits)} passages ({',
32         ↪ '.join(h.citation() for h in hits)})"],
33     }

```

The search query combines the question with what the ML agent found (the chosen estimator and any warnings), so the cited rules match the situation. Every hit carries its file and heading.

24 Policy agent

app/agents/policy_agent.py, lines 27–54

```

27 def run_policy_agent(state: AgentState, deps: Deps) -> AgentState:
28     constraints = state.get("constraints") or {}
29     memory = state.get("memory") or {}
30     exclude = set(memory.get("cooldown_ids", [])) | set(memory.get("excluded_ids", []))
31     exclude |= set(constraints.get("exclude_ids", []))
32
33     scored = load_scores(deps)
34     policy = (state.get("selection") or {}).get("policy", "net_value_top_k")
35     budget = int(state.get("budget") or deps.settings["agent"]["default_budget"])
36     if policy.startswith("tau"):
37         offers = rank_by_tau(scored, budget, exclude)
38     else:
39         offers = rank_by_net_value(scored, budget, exclude, constraints.get("min_net_value",
40         ↪ 0.0))

```

```

41 cols = ["customer_id", "rank", "tau_hat", "churn_risk", "value", "offer_cost",
    ↪ "net_value", "offer"]
42 records = offers[[c for c in cols if c in offers]].to_dict("records")
43 summary = policy_summary(offers)
44 summary.update({"budget": budget, "eligible_positive_net": int((scored["net_value"] >
    ↪ 0).sum()),
45                "excluded": len(exclude), "policy": policy})
46 return {
47     "offers": records,
48     "policy_summary": summary,
49     "campaign_id": state.get("campaign_id") or f"cmp-{time.strftime('%Y%m%d-%H%M%S')}",
50     "trace": state.get("trace", []) + [
51         f"policy_agent: {summary['n_offers']} offers (budget {budget}, "
52         f"{summary['eligible_positive_net']} with positive net value, {len(exclude)}"
    ↪ excluded)"]
53     ],
54 }

```

Exclusions come from three places: the cooldown list, earlier reviewer feedback, and constraints the reviewer added in this run.

25 Reviewer agent

app/agents/reviewer_agent.py, lines 18–64

```

18 def review(state: AgentState, settings: dict) -> dict:
19     blocking, fixable, bad_ids = [], [], set()
20     exp = state.get("experiment") or {}
21     exp_cfg = settings["experiment"]
22
23     if exp.get("srm_p", 1.0) < exp_cfg["srm_alpha"]:
24         blocking.append(f"SRM: arm sizes differ from the plan (p = {exp['srm_p']:.2g})")
25     if exp.get("max_abs_smd", 0.0) >= exp_cfg["smd_threshold"]:
26         blocking.append(f"Imbalance: max |SMD| = {exp['max_abs_smd']:.3f}")
27
28     model = state.get("model") or {}
29     allowed = (state.get("selection") or {}).get("estimators", [])
30     if model.get("method") and model["method"] not in allowed:
31         blocking.append(f"Estimator {model['method']} is not allowed for this data ({',
    ↪ '.join(allowed)})")
32     qini = (model.get("metrics") or {}).get("qini_true_normalized")
33     if qini is not None and qini <= 0:
34         blocking.append("Model ranks no better than random (normalized Qini <= 0)")
35
36     offers = state.get("offers") or []
37     memory = state.get("memory") or {}
38     cooldown = set(memory.get("cooldown_ids", [])) | set(memory.get("excluded_ids", []))
39     seen = set()
40     for o in offers:
41         cid = int(o["customer_id"])
42         if o["tau_hat"] <= 0:
43             fixable.append(f"customer {cid}: predicted effect <= 0 (sleeping dog or no
    ↪ effect)")
44             bad_ids.add(cid)
45         elif o["net_value"] <= 0:
46             fixable.append(f"customer {cid}: net value <= 0")
47             bad_ids.add(cid)
48         if cid in cooldown:
49             fixable.append(f"customer {cid}: contacted recently or excluded by a reviewer")
50             bad_ids.add(cid)
51         if cid in seen:
52             fixable.append(f"customer {cid}: listed twice")

```

```

53     seen.add(cid)
54     budget = int(state.get("budget") or settings["agent"]["default_budget"])
55     if len(offers) > budget:
56         fixable.append(f"{len(offers)} offers exceed the budget of {budget}")
57
58     return {
59         "passed": not blocking and not fixable,
60         "blocking": blocking,
61         "fixable": fixable[:20],
62         "n_fixable": len(fixable),
63         "bad_ids": sorted(bad_ids),
64     }

```

Blocking problems mean the evidence cannot be trusted: a failed SRM or balance check, an estimator the selector did not allow, or a model that ranks no better than random. Fixable problems concern individual offers: negative effect, negative net value, cooldown, duplicates, or too many offers. A human still approves every list through the API (Section 34).

26 Report

app/agents/report.py, lines 102–111

```

102 def run_report(state: AgentState, deps: Deps) -> AgentState:
103     provider = get_provider(deps.settings["agent"]["provider"])
104     template = build_text(state, deps)
105     facts = {k: state.get(k) for k in ("intent", "experiment", "policy_summary", "review",
106     ↪ "model")}
107     narrative = provider.narrate(facts, template)
108     return {
109         "narrative": narrative,
110         "provider": provider.name,
111         "trace": state.get("trace", []) + [f"report: written by {provider.name} provider"],
112     }

```

The brief is built from a template that uses only numbers from the state. An optional LLM may rephrase it.

27 Providers

The deterministic provider uses keyword rules for routing and the template for text, so tests and CI need no network. The optional Claude provider only rewrites the narrative, and any number it writes must already appear in the template:

app/agents/providers.py, lines 72–75

```

72 def numbers_match(text: str, reference: str) -> bool:
73     """Every number in `text` must also appear in `reference` (the template built from
74     ↪ state)."""
75     allowed = set(re.findall(r"-?\d+(?:[.]\d+)?", reference))
76     return all(n in allowed for n in re.findall(r"-?\d+(?:[.]\d+)?", text))

```

app/agents/providers.py, lines 49–69

```

49 def narrate(self, facts: dict[str, Any], template_text: str) -> str:
50     response = self.client.beta.messages.create(
51         model=self.model,
52         max_tokens=2000,

```

```

53     betas=["server-side-fallback-2026-07-01"],
54     fallbacks="default",
55     output_config={"effort": "low"},
56     system=(
57         "You write short briefs for a retention manager. Use only the facts given. "
58         "Keep every number exactly as written. Plain sentences, no headings, under 180
↪ words."
59     ),
60     messages=[{
61         "role": "user",
62         "content": "Facts (JSON):\n" + json.dumps(facts, default=str)
63         + "\n\nDraft to improve:\n" + template_text,
64     }],
65 )
66 if response.stop_reason == "refusal":
67     return template_text
68 text = "".join(b.text for b in response.content if b.type == "text").strip()
69 return text if numbers_match(text, template_text) else template_text

```

If the model refuses, adds a number, or the SDK is missing, the template text is used.

28 SQL service

app/services/sql_service.py, lines 49–58

```

49 def check_sql(sql: str) -> str:
50     """Accept one SELECT (or WITH ... SELECT) statement; reject anything else."""
51     stripped = sql.strip().rstrip(";").strip()
52     if ";" in stripped:
53         raise SQLGuardError("only one statement is allowed")
54     if not re.match(r"^(select|with)\b", stripped, re.I):
55         raise SQLGuardError("only SELECT queries are allowed")
56     if _FORBIDDEN.search(stripped):
57         raise SQLGuardError("query contains a forbidden keyword")
58     return stripped

```

Free-form SQL (only from an optional LLM path) must be one SELECT without write keywords, and then runs on a read-only connection. The tests try DROP, stacked statements, UPDATE, and PRAGMA.

29 Knowledge retrieval

app/services/knowledge.py, lines 54–63

```

54 def search(self, query: str, k: int = 3, min_score: float = 0.05) -> list[Hit]:
55     if self.matrix is None:
56         return []
57     sims = cosine_similarity(self.vectorizer.transform([query]), self.matrix).ravel()
58     order = sims.argsort()[::-1][:k]
59     return [
60         Hit(self.chunks[i][0], self.chunks[i][1], self.chunks[i][2], float(sims[i]))
61         for i in order
62         if sims[i] >= min_score
63     ]

```

The knowledge base is eight short markdown cards in docs/knowledge, split at \#\# headings. TF-IDF with cosine similarity is enough for a few dozen chunks and needs no embedding model.

30 Memory

app/services/memory.py, lines 56–63

```

56 def recently_contacted(self, days: int, now: datetime | None = None) -> set[int]:
57     """Customer ids contacted within the last `days` days (cooldown list)."""
58     since = ((now or utc_now()) - timedelta(days=days)).isoformat()
59     with self.store.connect() as conn:
60         rows = conn.execute(
61             "SELECT DISTINCT customer_id FROM contacts WHERE contacted_at >= ?", (since,)
62         ).fetchall()
63     return {int(r[0]) for r in rows}

```

Contacts are logged only after a human approves a campaign. That keeps the cooldown honest: a list that was rejected does not block those customers next month.

Part IV

Engineering

31 Pipeline

Each step is a plain function, so the command line, the tests, and Airflow run the same code.

pipelines/run_pipeline.py, lines 59–76

```

59 def step_generate(cfg) -> dict[str, Any]:
60     """Simulate the A/B test, an observational variant, and the current subscriber base."""
61     store = Store(cfg["paths"]["database"])
62     n, seed, sim = cfg["n_subscribers"], cfg["seed"], cfg["sim"]
63     exp = simulate(n, seed, sim, "rct")
64     obs = simulate(n, seed, sim, "observational")
65     cur = simulate(n, seed + CURRENT_SEED_OFFSET, sim, "rct")
66
67     # Tables the agent can query never contain the oracle columns.
68     store.write_table("experiment", exp.drop(columns=ORACLE_COLUMNS))
69     store.write_table("observational", obs.drop(columns=ORACLE_COLUMNS))
70     current = cur[["customer_id"] + FEATURES + ["value", "offer_cost"]]
71     store.write_table("current_subscribers", current)
72     # Oracle tables exist only for evaluation (and the agent eval), never for decisions.
73     store.write_table("oracle_experiment", exp[["customer_id"] + ORACLE_COLUMNS])
74     store.write_table("oracle_observational", obs[["customer_id"] + ORACLE_COLUMNS])
75     store.write_table("oracle_current", cur[["customer_id"] + ORACLE_COLUMNS])
76     return {"status": "success", "rows": {"experiment": len(exp), "observational": len(obs),
        ↪ "current": len(cur)}}

```

pipelines/run_pipeline.py, lines 97–110

```

97 def step_analyze(cfg) -> dict[str, Any]:
98     """SRM, balance, and the average effect. Stops the pipeline if the test is invalid."""
99     store = Store(cfg["paths"]["database"])
100     exp = store.read_table("experiment")
101     obs = store.read_table("observational").merge(store.read_table("oracle_observational"),
        ↪ on="customer_id")
102     result = analyze_experiment(

```

```

103     exp, feature_matrix(exp), cfg["experiment"]["srm_alpha"],
104     cfg["experiment"]["smd_threshold"], cfg["sim"]["treat_share"],
105 )
106 result["observational_check"] = observational_check(obs, feature_matrix(obs))
107 _save_json(result, _reports(cfg) / "experiment.json")
108 if not result["valid"]:
109     raise RuntimeError("experiment failed SRM or balance checks; stop before modeling")
110 return result

```

`step_analyze` raises an error when the test fails SRM or balance, which stops every later step.

32 Quality gates

pipelines/gates.py, lines 17–38

```

17 def evaluate_gates(experiment: dict[str, Any], bench: dict[str, Any]) -> dict[str, Any]:
18     table = bench["table"]
19     primary = bench["primary_model"]
20     lin = experiment["ate_lin"]
21
22     checks = {
23         # The A/B test itself must be valid before anything downstream counts.
24         "srm_passed": experiment["srm"]["passed"],
25         "balance_passed": experiment["balance"]["passed"],
26         "ate_ci_finite": _finite(lin["estimate"], lin["ci_low"], lin["ci_high"]),
27         # Qini check: the primary model ranks better than random and better than k-means.
28         "qini_beats_random": table[primary]["qini_true_normalized"]["mean"] > 0,
29         "qini_beats_kmeans": table[primary]["qini_true_normalized"]["mean"]
30         > table["kmeans"]["qini_true_normalized"]["mean"],
31         # Policy-value check: at the planned coverage, uplift targeting keeps at least
32         # as many customers as churn-risk targeting, on average over draws.
33         "policy_value_vs_risk": table[primary]["policy_true"]["mean"] >=
34         ↪ table["churn_risk"]["policy_true"]["mean"],
35         "no_nan_metrics": all(
36         ↪ _finite(cell["mean"]) for row in table.values() for cell in row.values() if cell
37         ↪ is not None
38     ),
39     }
40     return {"passed": all(checks.values()), "checks": checks}

```

These are sanity checks written before the first full run, not significance tests and not targets chosen after seeing results. The policy-value gate compares means over draws. A stricter rule (the lower bound of the gain CI above zero) would also pass today (4.0 customers), but it is not used as a gate.

33 MLflow registry

app/services/registry.py, lines 41–80

```

41 def log_and_register(
42     scorer_path: Path,
43     params: dict[str, Any],
44     metrics: dict[str, float],
45     report_path: Path | None,
46     cfg: dict,
47     promote: bool,
48 ) -> dict[str, Any]:
49     """Log one run, register the scorer as a new model version, promote if allowed."""

```

```

50 mlflow, client = _client(cfg)
51 name = cfg["mlflow"]["model_name"]
52 exp_name = cfg["mlflow"]["experiment"]
53 if client.get_experiment_by_name(exp_name) is None:
54     # Keep run artifacts next to the database, not in ./mlruns of whatever folder we run
    ↪ from.
55     artifact_root = Path(cfg["paths"]["artifacts"]) / "mlartifacts"
56     client.create_experiment(exp_name, artifact_location=artifact_root.resolve().as_uri())
57 mlflow.set_experiment(exp_name)
58 with mlflow.start_run() as run:
59     mlflow.log_params(params)
60     mlflow.log_metrics({k: float(v) for k, v in metrics.items() if v is not None})
61     mlflow.log_artifact(str(scorer_path), artifact_path="scorer")
62     if report_path and Path(report_path).exists():
63         mlflow.log_artifact(str(report_path), artifact_path="reports")
64     mlflow.set_tag("gates_passed", str(promote))
65
66 version = client.create_model_version(
67     name=ensure_model(client, name), source=f"runs:{run.info.run_id}/scorer",
    ↪ run_id=run.info.run_id
68 ).version
69
70 previous = None
71 if promote:
72     try:
73         previous = client.get_model_version_by_alias(name, CHAMPION).version
74     except Exception: # no champion yet
75         previous = None
76     if previous and previous != version:
77         client.set_registered_model_alias(name, PREVIOUS, previous)
78         client.set_registered_model_alias(name, CHAMPION, version)
79 return {"model_name": name, "version": str(version), "run_id": run.info.run_id,
80        "promoted": promote, "previous_champion": previous}

```

Every run is logged with its parameters, metrics, the scorer file, and the benchmark report. A new version becomes `champion` only if the gates passed; the old champion keeps the alias `previous_champion`. Rollback swaps the two aliases:

app/services/registry.py, lines 91–99

```

91 def rollback(cfg: dict) -> dict[str, Any]:
92     """Point `champion` back at `previous_champion`, and swap the two aliases."""
93     _, client = _client(cfg)
94     name = cfg["mlflow"]["model_name"]
95     current = client.get_model_version_by_alias(name, CHAMPION).version
96     previous = client.get_model_version_by_alias(name, PREVIOUS).version
97     client.set_registered_model_alias(name, CHAMPION, previous)
98     client.set_registered_model_alias(name, PREVIOUS, current)
99     return {"model_name": name, "champion": str(previous), "previous_champion": str(current)}

```

Serving loads models:/uplift_scorer@champion and falls back to the local file when MLflow is unreachable. Every API response that depends on a model reports the version.

34 API

The FastAPI service exposes health, model information, reload, experiment and benchmark reports, scoring, a plain ranking, the full agent, and campaign decisions. The decision endpoint is the human step:

app/api/main.py, lines 126–143

```

126 @app.post("/campaigns/{campaign_id}/decision")
127 def campaign_decision(campaign_id: str, req: ApproveRequest, deps: Deps = Depends(get_deps))
    ↪ -> dict:
128     """Human sign-off. Approval logs contacts (which start the cooldown); exclusions go to
    ↪ memory."""
129     if "pending_offers" not in deps.store.tables():
130         raise HTTPException(404, "no pending campaigns")
131     pending = deps.store.query("SELECT * FROM pending_offers WHERE campaign_id = ?",
    ↪ (campaign_id,))
132     if pending.empty:
133         raise HTTPException(404, f"campaign {campaign_id} not found")
134     deps.memory.remember("feedback", {"campaign_id": campaign_id, "reviewer": req.reviewer,
135                                     "approve": req.approve, "exclude_ids": req.exclude_ids,
    ↪ "note": req.note})
136     sent = 0
137     if req.approve:
138         to_send = pending[~pending["customer_id"].isin(req.exclude_ids)]
139         sent = deps.store.log_contacts(to_send, campaign_id, datetime.now(UTC).isoformat())
140     status = "approved" if req.approve else "rejected"
141     with deps.store.connect() as conn:
142         conn.execute("UPDATE pending_offers SET status = ? WHERE campaign_id = ?", (status,
    ↪ campaign_id))
143     return {"campaign_id": campaign_id, "status": status, "contacts_logged": sent}

```

35 UI

app/ui/streamlit_app.py has four tabs. *Experiment* shows SRM, balance, both ATE estimates, and the observational comparison. *Benchmark* shows the headline, the metric table, and Qini curves (observed or true). *Targeting* has a budget slider that compares the uplift list with a risk list of the same size. *Agent* runs the graph and shows the brief, the review, and the trace.

36 Docker

One image serves the API, the UI, the pipeline job, and an MLflow server. Dependencies are installed before the code is copied, so editing code does not reinstall packages.

infra/docker/Dockerfile

```

# One image for the API, the UI, and pipeline jobs. Code guide: section "Docker" (sec:docker).
FROM python:3.11-slim

# LightGBM needs the OpenMP runtime.
RUN apt-get update && apt-get install -y --no-install-recommends libgomp1 \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 MLFLOW_DISABLE_AGENT_HINT=1

# Install pinned dependencies first so code changes do not invalidate this layer.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY pyproject.toml README.md LICENSE ./
COPY app ./app
COPY pipelines ./pipelines
COPY scripts ./scripts
COPY configs ./configs

```

```
COPY docs/knowledge ./docs/knowledge
RUN pip install --no-cache-dir --no-deps -e .

EXPOSE 8000 8501
CMD ["uvicorn", "app.api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

37 Airflow

infra/airflow/dags/uplift_pipeline.py, lines 30–43

```
30 @dag(
31     dag_id="uplift_retention_weekly",
32     schedule="0 6 * * 1", # Mondays 06:00
33     start_date=pendulum.datetime(2026, 1, 1, tz="UTC"),
34     catchup=False,
35     max_active_runs=1,
36     default_args={"retries": 1, "retry_delay": timedelta(minutes=10)},
37     tags=["uplift", "retention", "causal"],
38 )
39 def uplift_retention_weekly():
40     steps = ["generate", "validate", "analyze", "benchmark", "train", "gates", "register",
41             ↪ "score", "agent"]
42     tasks = [task(task_id=s)(_run)(s) for s in steps]
43     for upstream, downstream in zip(tasks, tasks[1:], strict=False):
44         upstream >> downstream
```

The DAG runs every Monday. Each task calls one pipeline step, so the DAG holds no logic of its own and can be tested by running the steps directly.

38 CI

.github/workflows/ci.yml

```
# CI: lint, tests, a reduced pipeline run with quality gates, and the agent eval.
# A failed gate fails the build and the model is not promoted.
# Code guide: section "CI" (sec:ci).
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    env:
      AGENT_PROVIDER: deterministic
      MLFLOW_DISABLE_AGENT_HINT: "1"
      # Smaller job than the local run: same N, fewer draws, trees, and bootstrap reps.
      UPLIFT_DRAWS: "3"
      UPLIFT_TREES: "100"
      UPLIFT_BOOTSTRAP: "200"
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
```

```

python-version: "3.11"
cache: pip
- name: Install
  run: |
    pip install -r requirements.txt
    pip install --no-deps -e .
- name: Lint
  run: ruff check .
- name: Unit and integration tests (skipped when tests/ is not in the repo)
  run: if [ -d tests ]; then pytest -q; else echo "tests/ not in repository; skipping";
⇒ fi
- name: Pipeline with quality gates (Qini, policy value, experiment checks)
  run: python -m pipelines.run_pipeline
- name: Fail if any gate failed
  run: python -c "import json,sys; g=json.load(open('artifacts/reports/gates.json'));
⇒ print(g); sys.exit(0 if g['passed'] else 1)"
- name: Agent evaluation (golden cases)
  run: python -m pipelines.agent_eval
- name: MLflow rollback drill
  run: |
    python -m pipelines.run_pipeline --steps register
    python -m scripts.rollback_model --status
    python -m scripts.rollback_model
- uses: actions/upload-artifact@v4
  if: always()
  with:
    name: reports
    path: artifacts/reports/

```

CI runs lint, tests, a smaller pipeline (3 draws, 100 trees), fails on any gate, runs the agent evaluation, and practices a rollback. Locally the suite has 70 tests; the test folder is kept out of the public repository by choice, so on GitHub that step reports that it was skipped.

39 Agent evaluation

Nine golden cases in `evals/golden_cases.json` cover campaigns at two budgets, the experiment question, two method questions, a customer lookup, the cooldown after an approved campaign, and two injected failures the reviewer must catch. Each case runs against a copy of the database, so evaluation never changes real offers or memory.

`pipelines/agent_eval.py`, lines 55–89

```

55 def check(case: dict, state: dict, deps: Deps) -> list[str]:
56     """Return the list of failed expectations (empty = pass)."""
57     exp, fails = case["expect"], []
58     rev = state.get("review") or {}
59     if "intent" in exp and state.get("intent") != exp["intent"]:
60         fails.append(f"intent {state.get('intent')} != {exp['intent']}")
61     if "review_passed" in exp and rev.get("passed") != exp["review_passed"]:
62         fails.append(f"review passed = {rev.get('passed')}")
63     offers = pd.DataFrame(state.get("offers") or [])
64     if exp.get("offers_valid") and not offers.empty:
65         if (offers["tau_hat"] <= 0).any() or (offers["net_value"] <= 0).any():
66             fails.append("an offer has tau_hat <= 0 or net value <= 0")
67     if exp.get("within_budget") and len(offers) > case["budget"]:
68         fails.append(f"{len(offers)} offers > budget {case['budget']}")
69     if exp.get("beats_population_true_effect") and not offers.empty:
70         oracle = deps.store.read_table("oracle_current")
71         chosen = oracle[oracle["customer_id"].isin(offers["customer_id"])]["true_cate"].mean()
72         if not chosen > oracle["true_cate"].mean():
73             fails.append("chosen customers do not have a higher true effect than average")

```

```

74     if exp.get("no_recent_contacts") and not offers.empty:
75         contacted = set(deps.store.read_table("contacts")["customer_id"])
76         if contacted & set(offers["customer_id"]):
77             fails.append("offered to customers inside the cooldown window")
78     for text in exp.get("narrative_has", []):
79         if text not in (state.get("narrative") or ""):
80             fails.append(f"narrative missing '{text}'")
81     if "citation_has" in exp:
82         cites = " ".join(h["citation"] for h in state.get("knowledge") or [])
83         if exp["citation_has"] not in cites:
84             fails.append(f"no citation from {exp['citation_has']} (got: {cites})")
85     if "fixable_issue" in exp and not any(exp["fixable_issue"] in i for i in
86     ↪ rev.get("fixable", [])):
87         fails.append("expected fixable issue not raised")
87     if "blocking_issue" in exp and not any(exp["blocking_issue"] in i for i in
88     ↪ rev.get("blocking", [])):
89         fails.append("expected blocking issue not raised")
90     return fails

```

One check uses the simulation’s hidden truth: the customers the agent picks must have a higher true effect than the average current subscriber.

Part V

Putting it together

40 One customer’s journey

Take customer 42 in the current cohort, who pays \$8.99 a month (value \$75.52, offer cost \$2.80).

1. The pipeline scores the cohort with the champion causal forest: churn risk 6.3%, predicted churn reduction -0.48 pp.
2. Net value = predicted reduction $\times$ value $-$ cost = -3.16 dollars.
3. The policy agent only considers customers with a positive predicted effect and positive net value.
4. Asked directly (`python -m scripts.ask_agent "Should we send an offer to customer 42?"`), the agent routes to the customer path, looks up the score, retrieves the offer rules, and answers “do not contact”.

41 Limits and next steps

- **Everything rests on a simulator.** The 30-draw interval covers noise inside this simulator, not doubt about whether the simulator resembles a real business. The real-data results are weaker and mixed.
- **Individual effects are not well estimated.** Use the scores to rank, not as precise per-customer promises.
- **Net value is fragile.** With models trained on 25,000 rows the net-value policy lost money on average; with 50,000 rows one cohort made +1,638 dollars. A larger test, a cheaper offer,

or a better-targeted segment would change the economics.

- **Next:** doubly robust learners (DR-learner), calibration of $\hat{\tau}$ by decile, Thompson-style budget pacing, and a holdout A/B test of the uplift policy itself (the only way to confirm the gain on real customers).

42 Self-check questions

1. Why does ranking by churn risk send offers to lost causes? *Answer: high risk without the offer says nothing about whether the offer changes the outcome; lost causes have the highest risk and zero effect.*
2. Why is the Lin estimate the headline rather than the difference in means? *Answer: it was chosen in advance; picking whichever looks better afterwards is cherry-picking.*
3. Can Qini be computed on real data? PEHE? *Answer: Qini yes, it uses observed outcomes; PEHE no, it needs the true effect.*
4. Why does the DR loss rank models like PEHE? *Answer: its expectation is $PEHE^2$ plus a constant shared by all models.*
5. What does the bootstrap interval of the observed readout leave out? *Answer: training noise and model selection; models are frozen.*
6. Why are contacts logged only after approval? *Answer: so a rejected list does not put customers into cooldown.*

A Glossary

ATE	Average treatment effect: the mean effect over all customers.
CATE, uplift, $\tau(x)$	Conditional average treatment effect: the effect for customers with features x .
SRM	Sample ratio mismatch: arm sizes differ from the plan.
SMD	Standardized mean difference between arms for one feature.
Propensity $e(x)$	Probability of receiving the offer given features.
IPW / AIPW	(Augmented) inverse probability weighting. AIPW is doubly robust.
PEHE	Root mean squared error of the estimated effect.
Qini	Ranking metric: extra outcomes from targeting the top k versus random.
Policy value	Extra customers kept by a targeting rule.
Net value	$\hat{\tau} \times \text{value} - \text{cost}$.
pp	Percentage points.

B References

- Athey, S., Tibshirani, J., Wager, S. (2019). Generalized random forests. *Annals of Statistics*.
- Athey, S., Imbens, G. (2016). Recursive partitioning for heterogeneous causal effects. *PNAS*.
- Künzel, S., Sekhon, J., Bickel, P., Yu, B. (2019). Metalearners for estimating heterogeneous treatment effects using machine learning. *PNAS*.
- Lin, W. (2013). Agnostic notes on regression adjustments to experimental data. *Annals of Applied Statistics*.

- Radcliffe, N. (2007). Using control groups to target on predicted lift. *Direct Marketing Analytics Journal*.
- Kennedy, E. (2023). Towards optimal doubly robust estimation of heterogeneous causal effects. *Electronic Journal of Statistics*.
- Kohavi, R., Tang, D., Xu, Y. (2020). *Trustworthy Online Controlled Experiments*. Cambridge University Press (SRM).
- Diemert, E., Betlei, A., Renaudin, C., Amini, M.-R. (2018). A large scale benchmark for uplift modeling. *AdKDD*.
- Hillstrom, K. (2008). MineThatData E-Mail Analytics and Data Mining Challenge.
- Verhelst, T., et al. (2023). A churn prediction dataset from the telecom sector: a new benchmark for uplift modeling.

C Commands

```
python -m pipelines.run_pipeline                # full run (about 7 minutes)
python -m pipelines.run_pipeline --steps analyze,gates # selected steps
python -m scripts.ask_agent "Plan this month's retention campaign." --trace
python -m scripts.run_real_benchmarks           # needs data/external
python -m scripts.rollback_model --status
python -m pipelines.agent_eval
python docs/tools/build_manuais.py
```